Intrusion Detection for Industrial Control Systems

Rayhan Alcena*
Kyle Brindle*
ralce002@fiu.edu
kbrin013@fiu.edu
Florida International University
Miami, Florida, USA

Abstract

Industrial Control System (ICS) are critical infrastructure components that operate water treatment facilities, power grids, and manufacturing plants. The increasing number of cyberattacks targeting these systems that are attempting to manipulate sensor readings and actuator states, and network communications to disrupt physical processes. In this paper, we design and evaluate an anomaly-based intrusion detection system for ICS environments using the SWaT December 2019 dataset collected from an actual water treatment testbed. This dataset captures both sensor data and network traffic, enabling evaluation across two complementary detection surfaces. Our pipeline both implements and compares three unsupervised detection methods on both sensor and network data. The three approaches include, a Statistical Threshold baseline, an Isolation Forest, and an Autoencoder. The attack labels are gathered from the attack window, which covers two types of attacks: historian data exfiltration detected in network traffic, and sensor disruption detected in process data. We evaluated each model using precision, recall, F1-score, and AUC-ROC. Our findings demonstrate that neither network-based or sensor-based detection alone provides the full picture. It is necessary to combine both detection surfaces for a comprehensive ICS intrusion detection.

CCS Concepts

• **Computing methodologies** → **Machine learning**; **Artificial intelligence**; **Model development and analysis**; *Neural networks*; *Classification and regression trees*; • **Networks** → **Network protocols**; • **Security and privacy** → **Intrusion detection systems**.

Keywords

intrusion detection, ICS, anomaly detection, machine learning, classification, unsupervised learning, autoencoder, statistical threshold, isolation forest

ACM Reference Format:
Rayhan Alcena and Kyle Brindle. 2026. Intrusion Detection for Industrial Control Systems. In *Proceedings of CIS 5370*. ACM, New York, NY, USA, 7 pages.

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CIS 5370, Miami, FL
© 2026 Copyright held by the owner/author(s).

1 Introduction

Modern society relies on ever-growing infrastructure to serve and ever-growing population. Disruptions to this infrastructure can affect large numbers of people, making power plants, water treatment plants, and other critical facilities attractive targets for malicious actors such as hostile state-sponsored groups or ransomware operators seeking financial gain. As such, it is important to protect the control systems that manage this infrastructure from cyberattacks. However, the systems and protocols used in the Industrial Control Systems (ICS) that manage these potential targets are often not standardized [7], complicating the development of cybersecurity solutions that can protect this critical infrastructure. This necessitates the development of intrusion detection systems (IDS) that can operate on more generalized data while still preventing the compromise of an ICS.

This paper seeks to solve this problem by developing and testing an anomaly based IDS that analyzes sensor and network data independent of specific protocols and operating procedures. This data was acquired from the SWaT December 2019 dataset, provided by the Singapore University of Technology and Design and iTrust Labs[4]. The dataset covers 3 hours and 40 minutes of operations, including historian data exfiltration and sensor disruption attack windows, totaling 13,201 sensor readings and 20GB of network capture in the form of .pcap files. The .pcap files were transformed into 4,628,784 network flows using the software NFStream before being used.

This data was used to train Statistical Threshold, Isolation Forest, and Autoencoder models. The models were trained on the network and sensor data independently to create 6 models in total, 1 of each type for network and sensor data respectively. We then evaluated the models using precision, recall, F1-score, and AUC-ROC in order to determine their performance the potential of this concept to serve as an anomaly based IDS.

Contributions

- Developed a framework for analyzing network and sensor data and using it to train models for anomaly detection.
- Evaluated the performance of different models and insight into improvements for future works.

2 Background & Threat Model

2.1 SWaT Dataset

The December 2019 SWaT (Secure Water Treatment) dataset used is a subset of the larger dataset curated by the Singapore University of Technology and Design and iTrust Labs. Information from the dataset comes from a scaled water treatment testbed that models a six-stage water treatment process [3]. It was designed specifically to

allow researchers to observe interactions between component parts and simulate attacks with different levels of access to the system [3].The particular dataset used in this paper covers 3 hours and 40 minutes of operations with a historian data exfiltration attack carried out over 5 minutes, repeating 4 times with a 10 minute gap between attacks, and a sensor disruption attack carried out over 3 minutes, repeating 5 times with a 10 minute gap between attacks [4]. The data in the dataset included sensor and alarm data in a .csv file, and network captures in the form of .pcap files.

2.2 NFStream

The network capture files (.pcap) from the SWaT dataset were transformed into network flows using NFStream. NFStream is a network data analysis framework that turn individual packets into flows representing communication sessions based on the packet data such as IP address, ports, and protocols [2]. It also extracts packet and flow features into columns which can be more easily processed. In this paper, the network flows created were represented in a csv file.

2.3 Threat Model

The threat model for the SWaT dataset used has 2 components as per the log present in the dataset.

- **Exfiltrate historian data** - A Supervisory Control and Data Acquisition (SCADA) workstation is compromised using a USB thumbdrive with malware. This malware tries to exfiltrate stored historical operational data. It does so in 5 minute windows, repeating the attack 4 times, with a 10 minute gap from the end of the previous attack to the start of the next.
- **Disrupt sensor readings and processes** - A Supervisory Control and Data Acquisition (SCADA) workstation is compromised and downloads malware from a Command and Control server. This malware attempts to disrupt normal sensor operations. It does so in 3 minute windows, repeating the attack 5 times, with a 10 minute gap from the end of the previous attack to the start of the next.

3 System Design

3.1 System Overview

Our proposed system aims to address anomaly-based intrusion detection in ICS environments by using a pipeline consisting of multiple stages. These stages process our input, the SWaT December 2019 dataset, and generate outputs that identify sensor disruption attacks and network exfiltration activity.

3.2 Architecture and Workflow

The pipeline is made up of four components: data loading, preprocessing, anomaly detection, and evaluation. The data loading component loads raw network traffic, and sensor readings. The preprocessing component cleans and normalizes the data. Also, encodes the categorical alarm states as binary, and using the attack windows to label attack samples. Features are normalized using the Min-Max scaler to fit only on the training data. The training set consists of normal operation data, which mirrors the real-world deployment scenario where labeled attack data is scarce [6]. For the third component, anomaly detection, utilizes three unsupervised models: Statistical Threshold, Isolation Forest, and Autoencoder, independently to both network ad sensor data. Each of our models produce a per-sample anomaly score which is compared against a threshold to predict a binary value. Lastly, for the evaluation component, we compute the precision, recall F1-score, and AUC-ROC for all three models, which produces visualizations including confusion matrices, ROC curves, anomaly score distributions, and reconstruction error plots.

3.3 Design Rationale

Unsupervised anomaly detection was chosen because labeled attack data rarely shows up in real ICS deployments. The models are required to detect attack patterns it has never seen before during training [5]. Using only normal samples to for training ensures that the model is learning the expected behavior of the system. The three detection methods were chosen to compare the performance of a statistical baseline, a machine learning approach, and a deep learning method. This is all done to provide a comprehensive view of the detection landscape for the SWaT dataset. Evaluating both the network and sensor data separately allows our team to assess the coverage and limitations of each detection surface. Thus, motivating the case for combined detection [7].

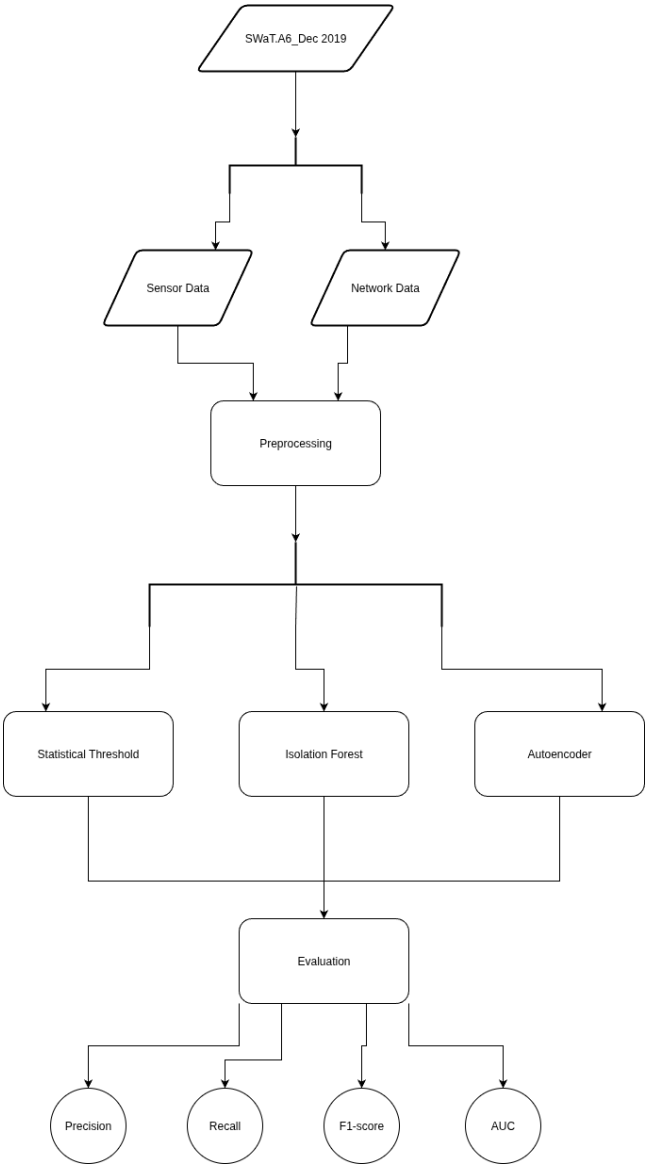

Figure 1: System pipeline: the SWaT December 2019 dataset is split into sensor and network data, preprocessed, and passed through three anomaly detection models. Each model is evaluated using precision, recall, F1-score, and AUC-ROC.

4 Implementation

For our implementation of the intrusion detection pipeline, we decided on a Jupyter notebook environment using Python version 3.12. Loading the data and preprocessing were mainly handled using the Pandas and NumPy libraries. Using Scikit-learn allowed us to implement both the Statistical Threshold and Isolation Forest models. The Autoencoder was implemented through the use of PyTorch. It is a symmetrical encoder-decoder architecture consisting of fully connected layers with Rectified Linear Unit (ReLU) activations, which was inspired by the reconstruction-based anomaly detection approaches in prior ICS research [1]. The encoder first compresses the 81 features down to a 16-dimensional bottleneck through intermediate layers of 64 and 32 units. The decoder is the reflection on the encoder's structure to reconstruct the original input. Training was performed for 50 epochs using the Adam optimizer with a learning rate of 0.001 and a Mean Squared Error (MSE) loss. We utilized an NVIDIA RTX GPU for GPU acceleration via CUDA in order to speed up the model's training time. Random seeds were fixed to ensure reproducibility. The detection thresholds, for the Autoencoder, were set to the at the percentile of training reconstruction errors that maximized F1-score on the test – 99th percentile for sensor data and 95th percentile for network data. Our Isolation Forest's contamination parameter was set to match the attack ratio of approximately 7% in the test set. Dependency management for the code base was handled by using the command-line tool uv. The full project environment is possible to reproduce by mean of the included `requirements.txt` file. All of our code/experiment is available in our GitHub repository at https://github.com/iamllcoolray/cis5370-final-project.

5 Evaluation

5.1 Evaluation Overview

We evaluated the intrusion detection pipeline on the SWaT December 2019 dataset. We divided the evaluation into two parts: network data evaluations and sensor data evaluations. Both parts of the evaluations follows the same structure. The training set consists of sample of normal operations recorded outside of the attack windows. The test set consisted of all samples of normal operations and the attacks that fall within the attack windows. Each detection model is trained exclusively on normal data and evaluated on the entire test set, which utilizes precision, recall, F1-score, and AUC-ROC. We used F1-score and AUC as the primary metrics. We had to ignore accuracy due to the heavily imbalanced dataset. For example, in the sensor dataset attacks only made up less than 7% of the test set. This imbalance could return an accuracy score of at least 93% in the event that the models predicted every sample as normal. This issue was less pronounced in the network dataset as, rather than having one data entry every second, it is based on network communications, multiple of which can happen at any given point in time. However, the dataset was still imbalanced enough to ignore accuracy, and will be discussed further in it's own section.

5.2 Network Data Evaluations

The test set used to evaluate the three detection models on the network data was comprised of 2,977,585 network flows, of which 769,483 fell during an attack window and were as such labeled as attack, meaning roughly 26% of the test set fell during an attack window and was labeled as such. It is also important to note that not all network data that happened during an attack window was malicious, as normal ICS network traffic continued during those timeframes, meaning that the models could classify a large number of seemingly false negatives, but still be distinguishing between normal and malicious traffic with respectable accuracy.

At first glance, all three models look to have a similar, mediocre performance, with precision, recall and f1 score, with the highest precision score belonging to the statistical threshold model at 0.5,

as well as recall and f1 scores below 0.33 for all models, which can be seen in Table 2.

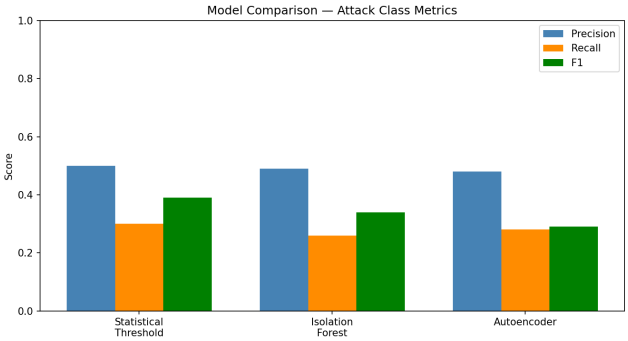

Figure 2: Precision, recall, and F1-score comparison across all three models on the network data.

The ROC curves and AUC scores in Figure 3 follow a similar trend as all three models show little variation in performance with AUC scores of 0.59, 0.61, and 0.61 for the Statistical Threshold, Isolation Forest, and Autoencoder, respectively.

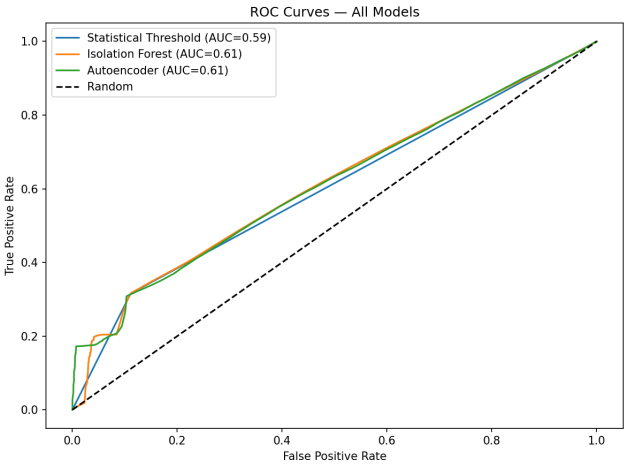

Figure 3: ROC curves for all three detection models on network data. All three models achieve similar AUC scores (0.59–0.61).

The confusion matrices in Figure 4 start to show something interesting. All three models perform well when it comes to correctly classifying non-attack network traffic. However, all three are classifying a large amount a traffic labeled "attack" as benign. As previously mentioned, for the network traffic section of the dataset all network traffic that occurred during an attack window was labeled "attack" even though it could be benign. As such, it is possible that the models performed much better than is indicated by their precision, recall, and f1-scores, with a large number of seemingly false negatives actually being correctly classified.

The anomaly score distribution present in Figure 5 display the same trend but provide little in the way of confirmation.

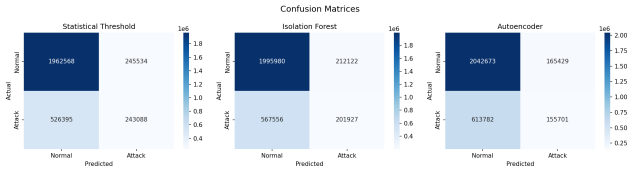

Figure 4: Confusion matrices for all three detection models on the network data.

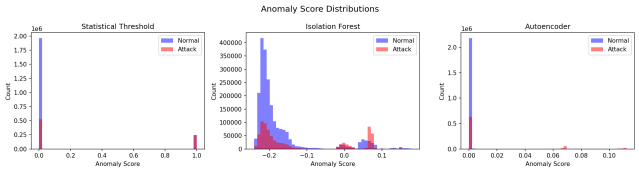

Figure 5: Anomaly score distributions for normal and attack samples on network data across all three models.

The plot of autoencoder reconstruction error over time lends validity to the suspicion that the the measures of precision, recall, and f1 score were misleadingly poor. There is a clear spike in autoencoder reconstruction error, meaning suspicious entries, during the attack windows. In particular the 4 areas shaded in red on the left hand side of the graph, representing the historian exfiltration attack windows, have much samples with high mean squared error(MSE). The 5 areas shaded in red on the right side of the graph correlate to the sensor disruption attack. There are some spikes the number of samples with high (MSE) but not nearly as obvious. If there is less network traffic associated with that attack then a higher proportion of normal network traffic would be flagged as "attack", influencing the precision, recall, F1, and AUC scores further. We believe that similar behavior was happening for all 3 models when dealing with network data, not just the autoencoder, and that the performance of the models was much better than their scores would lead one to believe at first glance.

Figure 6: Autoencoder reconstruction error (MSE) over time on the network data. Red shaded regions indicate the attack windows.

Table 1 summarizes the quantitative results for all three models on the non-attack network data. Table 2 summarizes the quantitative results for all three models on the network data labeled as attack.

Table 1: Network Data Non-Attack Detection Results

Model	Precision	Recall	F1	AUC
Statistical Threshold	0.79	0.89	0.84	0.59
Isolation Forest	0.78	0.90	0.84	0.61
Autoencoder	0.77	0.93	0.84	0.61

Table 2: Network Data Attack Detection Results

Model	Precision	Recall	F1	AUC
Statistical Threshold	0.50	0.32	0.39	0.59
Isolation Forest	0.49	0.26	0.34	0.61
Autoencoder	0.48	0.29	0.29	0.61

5.3 Sensor Data Evaluations

We evaluated the three detection models on the sensor data test set, which consists of 13,201 samples; 905 of these samples are labeled as attacks across five attack windows. Figure 7 shows us a summary of the precision, recall, and F1-score for each model on the attack class. The Autoencoder achieves the highest precision of 0.32, which means that when alerting, it is more than likely correct than the other two models. Unfortunately for us, its recall performed the worse out of the three models. This indicates that it misses a significant amount of attacks. The Isolation Forest balances these tradeoffs the most effectively. It has an F1-score of 0.31 and a recall of 0.75. Thus, making it the most suitable option for sensor based ICS intrusion detection in this evaluation.

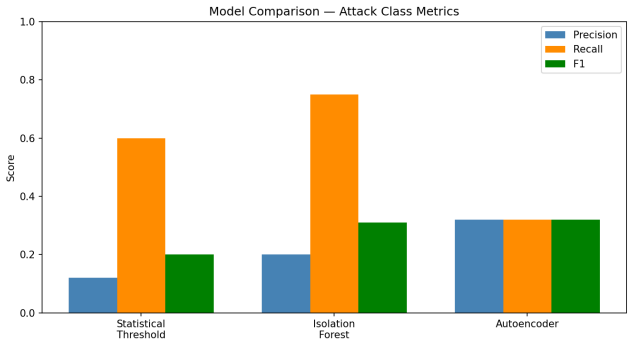

Figure 7: Precision, recall, and F1-score comparison across all three models on the sensor data.

Figure 8 shows the ROC curves for all three models. Here, the Isolation Forest produces the highest AUC of 0.83, which is then followed by the Autoencoder with an AUC of 0.76, and finally the Statistical Threshold at 0.69. Each model was able to perform well above random chance. This confirms that anomalous behavior is indeed detectable from process data alone.

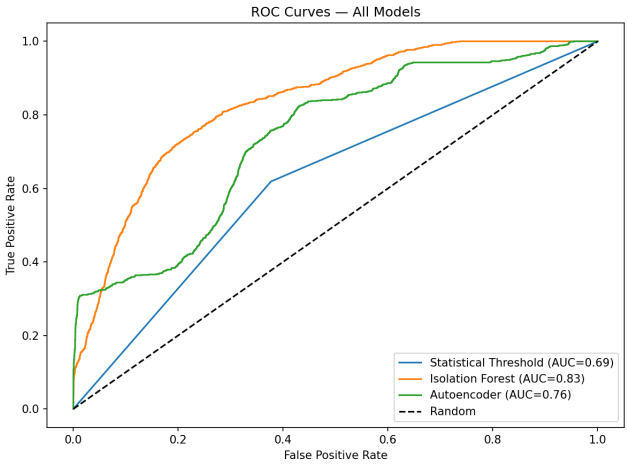

Figure 8: ROC curves for all three detection models on sensor data. All three models achieve similar AUC scores (0.69–0.83).

Figure 9 presents to us the confusion matrices for each model. The Statistical Threshold produces 4,635 false alarms while catching a total of 560 out of 905 attacks. While the Isolation Forest reduces the number of false alarms, by almost half, to 2,791 while only catching 677 attacks. Lastly, the Autoencoder produces the fewest false alarms at only 186, but catches the least number of attacks at 279.

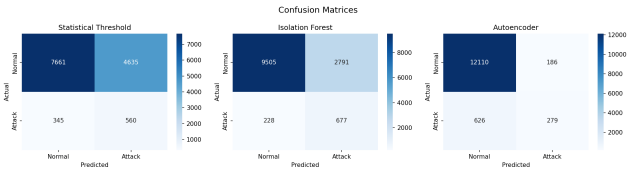

Figure 9: Confusion matrices for all three detection models on the sensor data.

Figure 10 displays the anomaly score distributions for both the normal and attack samples for each model. The Statistical Threshold produces a binary score distribution where zero equates to normal and one equals attack. There is heavy overlap between both sample types explaining its poor precision. Isolation Forest shows some separation between the normal and attack scores where the attack samples tend to have higher anomaly scores. The Autoencoder provides a cleaner separation at the lower reconstruction error values. The normal samples being closer to zero and the attack samples moving towards the higher error tail, which explains why it performed better in terms of precision.

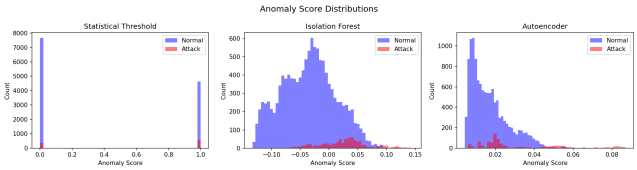

Figure 10: Anomaly score distributions for normal and attack samples on sensor data across all three models.

Figure 11 presents the Autoencoder reconstruction error over time on the test set. The five attack windows are highlighted in red. During windows 2 and 3, we can clearly observe the reconstruction error spiking, which confirms that the Autoencoder successfully detected these attack periods. During windows 1, 4, and 5, it shows some detection happening where much of the attack period falling well below the detection threshold. We do witness some elevated reconstruction error during normal operation, which would explain the false alarms observed in the confusion matrix.

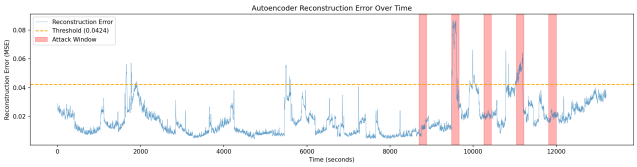

Figure 11: Autoencoder reconstruction error (MSE) over time on the sensor data. Red shaded regions indicate the attack windows.

Table 3 summarizes the quantitative results for all three models on sensor data.

Table 3: Sensor Data Detection Results

Model	Precision	Recall	F1	AUC
Statistical Threshold	0.12	0.62	0.20	0.69
Isolation Forest	0.20	0.75	0.31	0.83
Autoencoder	0.32	0.32	0.32	0.76

6 Related Work

There are multiple approaches to creating Intrusion Detection Systems for Industrial Control Systems in regards to how they deal with the unknown and obfuscated environments and protocols. For example, IPAL, created by Wolsing et al. [6], is an industrial protocol abstraction layer that attempts to divorce existing datasets and IDS approaches that are focused on specific industries, allowing them to be generalized and applied to domains other than the ones they were originally intended for. GeCos, created by Wolsing et al., similarly tries to create a generalized and widely applicable industrial IDS. However, it does so by automatically deriving appropriate models from historical data it has from appropriate domains. This paper differ from these approaches as our IDS will need to train on the ICS it is meant to protect, rather than from generalized datasets.

Abdelaty et al. took a different approach, and instead created DAICS, a deep learning IDS that is meant to be trained on data from the ICS it will be monitoring but constantly adapts to the state of the ICS as operations evolve over time. This is meant to prevent the IDS from needing updates from human intervention, as it will automatically adapt to changes in operating systems and procedure. Our approach differs by looking at network capture data, while this IDS focuses on the sensors and actuators associated with ICS sub-processes.

An approach fairly similar to ours is ZOE by Wressnegger et al. This IDS trains on large amounts of network data to learn the structure and content of messages and detect anomalies. Our IDS similarly looks rains on network data to find normal message features, but seperately trains on sensor data as well.

7 Discussion

7.1 Limitations

One limitation of this work, that we've discovered, is that the evaluation only focuses on an individual dataset, which was collected from a singular water treatment facility. Unfortunately, this may not generalize well across many different ICS environments. The attack windows, for the sensor data, totals up to about 15 minutes over the span of three and a half hour dataset. This makes the evaluation sensitive to the choice of threshold and limits how significant the statistical results are. As for the network data, the attacks windows contain much more benign data entries, limiting the effectiveness of the measures of precision, recall, F1-score, and AUC due to the amount of seemingly false negatives. In addition, the extracted features may not be able to fully capture the range of real-world threats. The pipeline does not function as a real-time streaming detector; limiting it from being applicable in live deployments.

7.2 Practical Considerations

Deploying this approach in a real-world ICS environment would require facing several challenges. The high false alarm rates, mainly exhibited by the Statistical Threshold and Isolation Forest models, would causes human operators to ignore them due to alert fatigue, which would reduce the value of the system [6]. Threshold tuning is highly sensitive and needs to be recalibrated for every facility and dataset. After our analysis of the network data, we also know that extracted network features need further feature engineering and better measures of performance than some of the more traditional ones to be useful in a real-world deployment. Also, an attacker with deep understanding of the detection models could in theory cook up slow/gradual manipulations that could stay within the bounds of normal behavior on both network and sensor levels, avoiding anomaly-based detection almost entirely [5].

7.3 Future Work

With more time, future research could explore many new ideas that could enhance our detection capabilities. First, merging our network and sensor detectors into a unified detection system would provide us with end-to-end coverage of both types of attacks observed in this dataset. Improved network feature engineering, such as incorporating protocol-level features using tools like IPAL [6], could help boost network-based detection performance beyond

what raw traffic statistics provide. Additionally, testing the pipeline on other datasets, such as WaDi, to evaluate the generalization across different ICS environments [7]. Implementing a real-time streaming detector could actually make the system deployable in live settings for both network and sensor streams at the same time. Finally, including temporal models, such as LSTM-Autoencoders could improve detecting attacks that develop slowly over time on both data sources [1].

8 Collaboration Table

Table 4: Project Collaboration Table

Name	Writing Tasks	Coding Tasks
Rayhan Alcena	Abstract, CCS Concepts, System Design, Implementation, Evaluation, Sensor Data Evaluations, Discussion, Collaboration Table, Conclusion	Sensor data preprocessing, sensor-based anomaly detection, attack labeling, Statistical Threshold, Isolation Forest, Autoencoder implementation, evaluation and visualizations
Kyle Brindle	Introduction, Background/Threat Model, Evaluation, Network Data Evaluations, Related Work, Collaboration Table, Conclusion	Network data preprocessing, Network pcap processing, network-based anomaly detection, attack labeling, Statistical Threshold, Isolation Forest, Autoencoder implementation, evaluation and visualizations

9 Conclusion

This paper presented an anomaly-base intrusion detection pipeline for ICS environments on the SWaT December 2019 dataset. We implemented and compared three unsupervised detection models: Statistical Threshold, Isolation Forest, and Autoencoder. They were applied independently to both process sensor data and network traffic data. Attack labels were created directly from the attack windows. This covers two attack types: sensor disruption observed in process data, and historian data exfiltration observed in network traffic.

Our evaluation demonstrates that the two detection surfaces exhibit vastly different performance characteristics. Regarding sensor data, Isolation Forest was able to achieve the strongest overall detection with an AUC of 0.83 and a recall of 0.75. The Autoencoder was able to produce the best precision score of 0.32 with the fewest false alarms. Now on the network data, all threes of our models were able to perform similarly with AUC scores between 0.59 and 0.61. The Statistical Threshold held the best F1-score of 0.39. However, these scores are possibly misleading due to a high amount of benign data being present in attack windows and being labeled as such. This modest display of performance on the network data highlights how difficult it is to measure the success of classifying network data without full context.

A great discovery is that both the sensor-based and network-based detections are complementary, neither method alone can capture all attacks. These results emphasize that comprehensive ICS intrusion detection requires monitoring both the physical process and the communication network at the same time.

References

[1] Maged Abdelaty, Roberto Doriguzzi-Corin, and Domenico Siracusa. 2021. DAICS: A Deep Learning Solution for Anomaly Detection in Industrial Control Systems. *IEEE Transactions on Emerging Topics in Computing* 10, 1117–1129. doi:10.48550/arXiv.2009.06299

[2] Zied Aouini and Adrian Pekar. 2022. NFStream: A Flexible Network Data Analysis Framework. *Computer Networks* 204 (2022). doi:10.1016/j.comnet.2021.108719

[3] Aditya P. Mathur and Nils Ole Tippenhauer. 2016. SWaT: A Water Treatment Testbed for Research and Training on ICS Security. In *Proceedings of the 2016 International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater).* 31–36. doi:10.1109/CySWater.2016.7469060

[4] Siddhant Shrivastava, Francisco Furtado, and Desmond Wan. 2019. SWaT Dataset December 2019. iTrust, Singapore University of Technology and Design. https://itrust.sutd.edu.sg/itrust-labs_datasets/dataset_info/

[5] Konrad Wolsing, Eric Wagner, Luisa Lux, Klaus Wehrle, and Martin Henze. 2025. GeCos Replacing Experts: Generalizable and Comprehensible Industrial Intrusion Detection. In *Proceedings of the 34th USENIX Security Symposium.* Seattle, WA, USA. https://www.usenix.org/conference/usenixsecurity25/presentation/wolsing

[6] Konrad Wolsing, Eric Wagner, Antoine Saillard, and Martin Henze. 2022. IPAL: Breaking up Silos of Protocol-dependent and Domain-specific Industrial Intrusion Detection Systems. In *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2022).* Limassol, Cyprus, 1–16. doi:10.1145/3545948.3545968

[7] Christian Wressnegger, Ansgar Kellner, and Konrad Rieck. 2018. ZOE: Content-Based Anomaly Detection for Industrial Control Systems. In *Proceedings of the 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN).* Luxembourg City, Luxembourg, 127–138. https://intellisec.de/pubs/2018-dsn.pdf